

Distributed Word Representations and Word2Vec

1.1 Chapter Overview and Learning Outcomes

In Chapter ??, we analyzed how classical statistical language models struggle under the combinatorial curse of dimensionality (V^n) and the complete lack of semantic overlap between distinct vocabulary symbols. In Chapter ??, we proved that neural networks can serve as universal function approximators when provided with continuous vector inputs and non-linear activations.

This chapter bridges the gap between discrete linguistic symbols and continuous neural architectures. We examine the geometric failure modes of traditional one-hot encodings, formalize the Distributional Hypothesis, and construct distributed vector representations. We present the complete mathematical matrix formulation of Continuous Bag-of-Words (CBOW) and Continuous Skip-Gram [?], derive the computational bottlenecks of full Softmax, and provide step-by-step gradient derivations for Hierarchical Softmax and Skip-Gram Negative Sampling (SGNS) [?]. We analyze the subword morphology mechanics of FastText [?] for out-of-vocabulary (OOV) tokens and explore the geometric structure of semantic vector spaces via Principal Component Analysis (PCA) and linear relational translations.

Upon completing this chapter, you will be able to:

- Quantify the geometric pathologies of one-hot vectors: extreme sparsity, curse of dimensionality, pairwise orthogonality ($\cos = 0$), and uniform Euclidean distance ($L_2 = \sqrt{2}$).
- Explain the paradigm shift from local to distributed representations and articulate the Distributional Hypothesis (Harris, 1954; Firth, 1957).
- Formalize the self-supervised cloze prediction task and trace the 7-step embedding training loop.
- Formulate CBOW mathematically using input projection matrix $\mathbf{W}_{\text{in}}$, output matrix $\mathbf{W}_{\text{out}}$, binary context presence vector $\mathbf{e}$, hidden state $\mathbf{h}$, and softmax logits.
- Derive the computational complexity of standard Softmax ($O(V)$) and contrast it with Hierarchical Softmax ($O(\log_2 V)$) and Negative Sampling ($O(K)$).
- Derive the analytical gradients of the Skip-Gram Negative Sampling (SGNS) loss with respect to center word, context word, and noise vectors.
- Explain the mathematical motivation behind the unigram noise distribution exponent $\frac{3}{4} = 0.75$.
- Break down the character n -gram subword composition of FastText and explain how it solves Out-of-Vocabulary (OOV) tokens and captures morphology.
- Interpret vector arithmetic in continuous semantic spaces (e.g., $\vec{v}_{\text{Germany}} - \vec{v}_{\text{Berlin}} \approx \vec{v}_{\text{capital_of}}$).

1.2 The Geometry of One-Hot Representations

In traditional computational linguistics and categorical modeling, a discrete vocabulary $\mathcal{W}$ containing $|\mathcal{W}| = V$ unique words establishes an arbitrary lexicographic index:

$$\mathcal{W} = \{w_1, w_2, \dots, w_V\}. \quad (1.1)$$

For example, consider a toy vocabulary of $|\mathcal{W}| = 5$ tokens:

$$\mathcal{W} = \{\text{dog}, \text{cat}, \text{fish}, \text{pen}, \text{pencil}\}. \quad (1.2)$$

Under *one-hot encoding*, each word $w_i \in \mathcal{W}$ is mapped to an independent standard basis vector $\mathbf{e}_i \in \{0, 1\}^V$ in V -dimensional Euclidean space $\mathbb{R}^V$, where the i -th entry is 1 and all remaining $V - 1$ entries are 0:

1. dog $\implies \mathbf{e}_1 = [1, 0, 0, 0, 0]^\top$
2. cat $\implies \mathbf{e}_2 = [0, 1, 0, 0, 0]^\top$
3. fish $\implies \mathbf{e}_3 = [0, 0, 1, 0, 0]^\top$
4. pen $\implies \mathbf{e}_4 = [0, 0, 0, 1, 0]^\top$
5. pencil $\implies \mathbf{e}_5 = [0, 0, 0, 0, 1]^\top$

1.2.1 Pathologies and Failure Modes of One-Hot Vectors

While one-hot encoding provides a straightforward numeric representation, it introduces severe mathematical pathologies that cripple neural optimization:

1. Extreme Sparsity and the Curse of Dimensionality. In practical NLP, a realistic vocabulary contains $V \geq 50,000$ to 100,000 words. Storing a single token requires a 50,000-dimensional vector. For each vector, $V - 1 = 49,999$ coordinates are zero, meaning over 99.99% of the memory represents uninformative empty space. The representation does not scale to large corpora.

2. Strict Pairwise Orthogonality and Equidistance. The most fatal limitation of one-hot representations is their complete geometric neutrality.

Theorem 1.1: Pairwise Orthogonality and Equidistance of One-Hot Vectors

Let $\mathbf{e}_i, \mathbf{e}_j \in \{0, 1\}^V$ be the one-hot representations of any two distinct words $w_i, w_j \in \mathcal{W}$ with $i \neq j$. Their cosine similarity is identically zero:

$$\cos(\mathbf{e}_i, \mathbf{e}_j) = \frac{\langle \mathbf{e}_i, \mathbf{e}_j \rangle}{\|\mathbf{e}_i\|_2 \|\mathbf{e}_j\|_2} = \frac{\sum_{k=1}^V e_{ik} e_{jk}}{\sqrt{1} \sqrt{1}} = 0, \quad \forall i \neq j. \quad (1.3)$$

Furthermore, their pairwise Euclidean distance is identically $\sqrt{2}$:

$$L_2(\mathbf{e}_i, \mathbf{e}_j) = \|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{\sum_{k=1}^V (e_{ik} - e_{jk})^2} = \sqrt{1^2 + (-1)^2} = \sqrt{2}, \quad \forall i \neq j. \quad (1.4)$$

💡 Intuition & Analogy: The Semantic Blindness of Orthogonal Space

Under one-hot encoding, every pair of words is situated at exactly the same geometric distance ($L_2 = \sqrt{2}$) and oriented at a right angle ($\theta = 90^\circ$). The representation asserts that:

$$\text{distance}(\text{pen}, \text{pencil}) = \text{distance}(\text{pen}, \text{dog}) = \sqrt{2}.$$

Syntactically and semantically related words are as completely separated as totally unrelated concepts. One-hot vectors form an orthogonal coordinate frame that cannot capture synonymy, hierarchy, or grammatical relatedness.

1.3 Distributed Representations and the Distributional Hypothesis

1.3.1 Local vs. Distributed Representations

One-hot encoding is a classic **local representation**: each discrete concept is assigned an isolated, single-coordinate identifier. If coordinate i fails, all information about word i vanishes.

By contrast, **distributed representations** (pioneered by Geoffrey Hinton [?]) distribute semantic concepts across d continuous latent dimensions (where $d \ll V$, typically $d \in [100, 300]$ for Word2Vec, and $d \in [768, 4096]$ for modern LLMs). Each dimension represents an abstract, distributed feature, and each concept is defined by a dense vector of continuous values:

$$\mathbf{v}_w = [v_1, v_2, \dots, v_d]^\top \in \mathbb{R}^d. \quad (1.5)$$

1.3.2 The Distributional Hypothesis

How can a machine learn meaningful continuous coordinates for words without human annotations? Modern NLP relies on the *Distributional Hypothesis*, formulated by linguists Zellig Harris (1954) and John Rupert Firth (1957):

“You shall know a word by the company it keeps.” (J.R. Firth, 1957)

Words that appear in similar textual contexts tend to possess similar semantic and pragmatic meanings. If “*pen*” and “*pencil*” regularly co-occur with verbs like “*write*”, “*draw*”, and nouns like “*essay*”, “*paper*”, their learned continuous coordinates should be placed close together in vector space.

1.3.3 Framing the Self-Supervised Prediction Task

To learn distributed representations automatically from unannotated text corpora, we formulate an auxiliary prediction task: *the cloze test* (fill-in-the-blank). Given an observed context window of surrounding tokens, the model estimates the conditional probability that a candidate word w fills the missing position:

$$\mathbb{P}(x_t = w \mid \text{context}) = \mathbb{P}(x_t = w \mid x_{t-c}, \dots, x_{t-1}, x_{t+1}, \dots, x_{t+c}). \quad (1.6)$$

For example, given the sentence:

“I used a [PENCIL] to write the essay”

the model is tasked with assigning high probability mass to plausible instruments (“*pencil*”, “*pen*”) while assigning low probability to incongruous entities (“*cat*”, “*fish*”).

1.3.4 The 7-Step Embedding Training Loop

Slides 7 and 8 detail the iterative mechanics used to solve this self-supervised task:

1. **Step 1 (Input Initialization):** Assign each context word to a continuous vector $\mathbf{v} \in \mathbb{R}^d$ (initialized randomly from a small Gaussian distribution).
2. **Step 2 (Context Aggregation):** Aggregate all active context word vectors into a single summary vector $\mathbf{h} \in \mathbb{R}^d$ (typically via element-wise summation or averaging).
3. **Step 3 (Output Initialization):** Assign each candidate target word in vocabulary $\mathcal{W}$ to an output vector $\mathbf{u} \in \mathbb{R}^d$ (also initialized randomly).
4. **Step 4 (Similarity Scoring):** Measure the alignment between context representation $\mathbf{h}$ and all candidate output vectors $\mathbf{u}_w$ via the inner product (dot product):

$$\hat{p}_w = \mathbf{h}^\top \mathbf{u}_w.$$

5. **Step 5 (Probability Normalization):** Apply the softmax function over all V candidate scores to obtain a valid categorical distribution $\mathbf{p} \in \mathbb{R}^V$.
6. **Step 6 (Loss Evaluation):** Evaluate the prediction against the ground-truth target word using Categorical Cross-Entropy loss: $\mathcal{L} = -\log p_{\text{target}}$.
7. **Step 7 (Gradient Backpropagation):** Compute analytical gradients with respect to both input and output word vectors, adjusting their coordinates via Gradient Descent.

Rinse and Repeat across Millions of Sentences. This exact optimization cycle is repeated across billions of sentences in large text corpora:

*“I used a **pencil** to write the essay”*
*“You used my **pen** to write a letter”*

Because “pen” and “pencil” share virtually identical surrounding context tokens (“used”, “to write”), the gradient updates nudge their respective vectors $\mathbf{v}_{\text{pen}}$ and $\mathbf{v}_{\text{pencil}}$ toward the exact same geometric neighborhood in $\mathbb{R}^d$.

1.4 Mathematical Matrix Formulation of Continuous Bag of Words (CBOW)

In Slides 9 and 10, the Continuous Bag of Words (CBOW) architecture is formalized in terms of matrix algebra, laying out the exact tensor dimensions:

1. Input Embedding Matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$. The input embedding matrix contains the continuous representations of all V vocabulary tokens when they appear in the context. Each column $j \in \{1, \dots, V\}$ corresponds to the d -dimensional input vector $\mathbf{v}_j \in \mathbb{R}^d$ of word w_j :

$$\mathbf{W}_{\text{in}} = \begin{bmatrix} | & | & & | \\ \mathbf{v}_1 & \mathbf{v}_2 & \dots & \mathbf{v}_V \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{d \times V}. \quad (1.7)$$

2. Output Embedding Matrix $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$. The output embedding matrix contains the continuous representations of all V vocabulary tokens when they appear as prediction targets. Each column $k \in \{1, \dots, V\}$ corresponds to the d -dimensional target vector $\mathbf{u}_k \in \mathbb{R}^d$ of word w_k :

$$\mathbf{W}_{\text{out}} = \begin{bmatrix} | & | & & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \dots & \mathbf{u}_V \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{d \times V}. \quad (1.8)$$

3. Context Binary Presence Vector $\mathbf{e} \in \{0, 1\}^{V \times 1}$. The surrounding context is encoded as a multi-hot binary indicator vector $\mathbf{e} \in \{0, 1\}^V$, where entry $e_j = 1$ if word w_j is present in the context window, and $e_j = 0$ otherwise:

$$\mathbf{e} = [1, 0, 0, 1, \dots, 0, 1]^\top \in \{0, 1\}^V. \quad (1.9)$$

Note on Bag-of-Words: Storing context as a binary presence vector sums word vectors indiscriminately, discarding the relative sequential ordering of words.

4. Hidden Representation $\mathbf{h} \in \mathbb{R}^{d \times 1}$. Multiplying $\mathbf{W}_{\text{in}}$ by context vector $\mathbf{e}$ accumulates the input vectors of all active context tokens:

$$\underbrace{\mathbf{h}}_{[d \times 1]} = \underbrace{\mathbf{W}_{\text{in}}}_{[d \times V]} \underbrace{\mathbf{e}}_{[V \times 1]} = \sum_{j \in \text{context}} \mathbf{v}_j. \quad (1.10)$$

Here, the binary vector $\mathbf{e}$ acts as an efficient multi-column selector across $\mathbf{W}_{\text{in}}$.

5. Unnormalized Similarity Score Logits $\hat{\mathbf{p}} \in \mathbb{R}^{1 \times V}$. To determine which vocabulary word is most compatible with context $\mathbf{h}$, the model computes inner products between $\mathbf{h}$ and all columns of $\mathbf{W}_{\text{out}}$:

$$\underbrace{\hat{\mathbf{p}}}_{[1 \times V]} = \underbrace{\mathbf{h}^\top}_{[1 \times d]} \underbrace{\mathbf{W}_{\text{out}}}_{[d \times V]} = [\mathbf{h}^\top \mathbf{u}_1, \mathbf{h}^\top \mathbf{u}_2, \dots, \mathbf{h}^\top \mathbf{u}_V]. \quad (1.11)$$

Each scalar logit $\hat{p}_k = \mathbf{h}^\top \mathbf{u}_k$ measures the angular alignment and magnitude compatibility between the context state $\mathbf{h}$ and output word w_k .

6. Normalized Categorical Probability Distribution $\mathbf{p} \in \mathbb{R}^{1 \times V}$. Applying the softmax transformation exponentiates the logits and normalizes them to sum to unity:

$$\mathbf{p} = \text{softmax}(\hat{\mathbf{p}}) \in \mathbb{R}^{1 \times V}, \quad \text{where } p_i = \frac{\exp(\hat{p}_i)}{\sum_{j=1}^V \exp(\hat{p}_j)} = \frac{\exp(\mathbf{h}^\top \mathbf{u}_i)}{\sum_{j=1}^V \exp(\mathbf{h}^\top \mathbf{u}_j)}. \quad (1.12)$$

7. Loss Backpropagation. Given one-hot ground-truth target vector $\mathbf{y} \in \{0, 1\}^V$ corresponding to true word w_c , Categorical Cross-Entropy evaluates to:

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^V y_k \log p_k = - \log p_c = -\mathbf{h}^\top \mathbf{u}_c + \log \left(\sum_{j=1}^V \exp(\mathbf{h}^\top \mathbf{u}_j) \right). \quad (1.13)$$

Reverse-mode automatic differentiation propagates gradients from $\mathcal{L}_{\text{CE}}$ through $\hat{\mathbf{p}}$ and $\mathbf{h}$ into both embedding matrices $\mathbf{W}_{\text{in}}$ and $\mathbf{W}_{\text{out}}$, adjusting coordinates via SGD.

Figure 1.1 illustrates this end-to-end matrix computation.

1.5 The Word2Vec Architectures: CBOW vs. Skip-Gram

In 2013, Tomas Mikolov and colleagues at Google [??] introduced **Word2Vec**, demonstrating that distributed word embeddings could be trained efficiently on billions of tokens in hours using log-linear models.

Word2Vec comprises two complementary architectural formulations:

1. **Continuous Bag-of-Words (CBOW):** Uses the surrounding context words $\{w_{t-c}, \dots, w_{t+c}\} \setminus \{w_t\}$ to predict the center target word w_t .

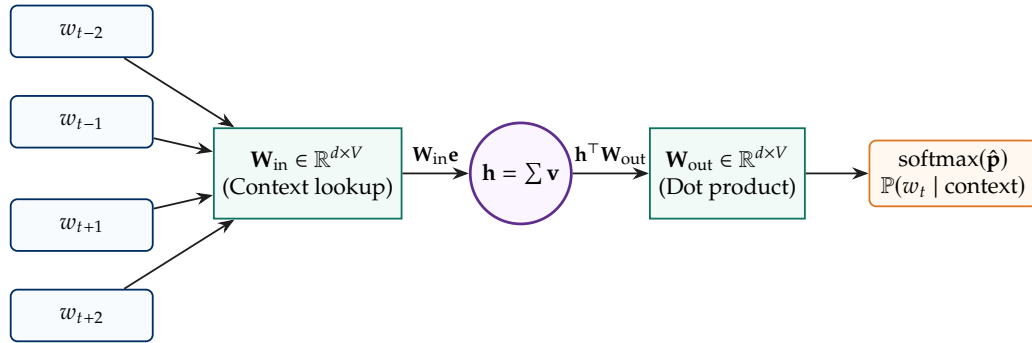


Figure 1.1: Continuous Bag-of-Words (CBOW) architecture. Context words $w_{t-c}, \dots, w_{t+c}$ are looked up in $\mathbf{W}_{\text{in}}$ and summed to form context state $\mathbf{h} \in \mathbb{R}^d$. Inner products with output matrix $\mathbf{W}_{\text{out}}$ yield similarity logits normalized via softmax to predict the center target token w_t .

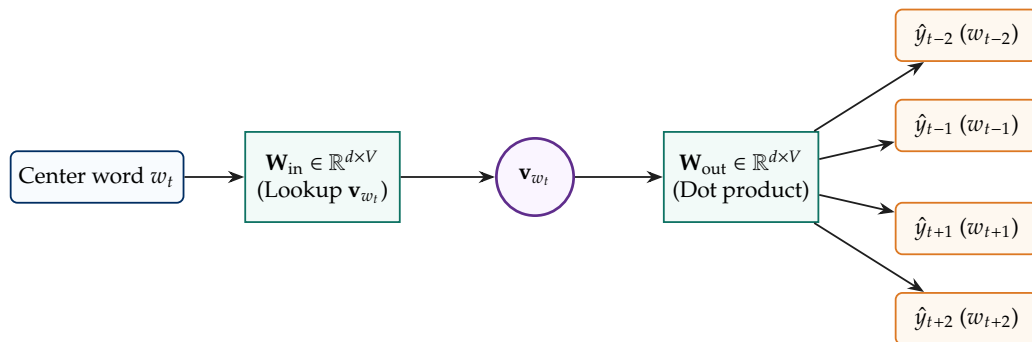


Figure 1.2: Continuous Skip-Gram architecture. A single center target word w_t is projected via $\mathbf{W}_{\text{in}}$ into vector $\mathbf{v}_{w_t} \in \mathbb{R}^d$, which predicts the probability distribution of context words across window radius c through output matrix $\mathbf{W}_{\text{out}}$.

2. **Continuous Skip-Gram:** Inverts the CBOW objective: uses the single center target word w_t to predict each of the surrounding context words within radius c .

Definition 1.1: *Skip-Gram Objective Function*

Given a training sequence of T words $w_1, w_2, \dots, w_T$ and a context window of half-width c , the Skip-gram model maximizes the average log-likelihood:

$$\mathcal{L}_{SG} = \frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log \mathbb{P}(w_{t+j} | w_t). \quad (1.14)$$

Under standard multinomial softmax, the conditional probability of context word w_O given center word w_I is:

$$\mathbb{P}(w_O | w_I) = \frac{\exp(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I})}{\sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{v}_{w_I})}, \quad (1.15)$$

where $\mathbf{v}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{in}$ (input embedding) and $\mathbf{u}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{out}$ (output context embedding).

Bengio (2000/2003) vs. Mikolov (2013). Slide 11 notes that Yoshua Bengio et al. [?] had previously presented a similar continuous distributed approach in 2000. However, Bengio's model enforced strict *causality* (predicting token w_t exclusively from past tokens w_{t-1}, w_{t-2}) and included a computationally heavy non-linear hidden layer in the MLP. Mikolov eliminated the non-linear hidden layer entirely, resulting in log-linear architectures that could be trained on orders of magnitude more data.

1.6 Scaling Softmax: Hierarchical Softmax and Negative Sampling

1.6.1 The Softmax Computational Bottleneck

Evaluating Equation (1.15) requires summing $\exp(\mathbf{u}_w^\top \mathbf{v}_{w_I})$ across every word $w \in \{1, \dots, V\}$ in the denominator. For realistic vocabularies ($V = 10^5$ to 10^6) and embedding dimensions $d = 300$:

- Computing the denominator requires $O(V \cdot d)$ operations per token step.
- Across a corpus of 10^9 tokens, computing the normalizer requires $10^9 \times 10^5 \times 300 = 3 \times 10^{16}$ floating-point operations.

This $O(V)$ scaling renders naive softmax training completely intractable. Mikolov et al. [?] introduced two mathematical workarounds: *Hierarchical Softmax* and *Negative Sampling*.

1.6.2 Workaround 1: Hierarchical Softmax ($O(\log_2 V)$)

Hierarchical Softmax evaluates the probability distribution using a binary tree structure (typically a Huffman tree, where frequent words are placed near the root). The V vocabulary words form the leaf nodes of the tree.

For each leaf word w , there exists a unique path from the root node to w of length $L(w)$. At each internal node j along the path, the model computes the probability of taking the left versus

right branch using a sigmoid classifier:

$$\mathbb{P}(w | w_I) = \prod_{j=1}^{L(w)-1} \sigma \left(\pm \mathbf{n}(w, j)^\top \mathbf{v}_{w_I} \right), \quad (1.16)$$

where $\mathbf{n}(w, j)$ is the parameter vector associated with internal node j . Because the expected depth of a balanced Huffman tree is $\mathcal{O}(\log_2 V)$, evaluating $\mathbb{P}(w | w_I)$ drops from $\mathcal{O}(V)$ to $\mathcal{O}(\log_2 V)$. For $V = 100,000$, $\log_2(100,000) \approx 17$ operations, achieving a 5,800× speedup.

1.6.3 Workaround 2: Skip-Gram with Negative Sampling (SGNS)

Rather than evaluating a normalized probability distribution over all V words, **Negative Sampling** reformulates the multiclass prediction task as a collection of independent binary logistic classification problems:

Given a word pair (w_I, w) , classify whether w is a genuine context word ($y = 1$) or drawn from random noise ($y = 0$).

Definition 1.2: Skip-Gram with Negative Sampling (SGNS) Objective

For each true center-context pair (w_I, w_O) , the model generates K fictitious “negative” noise words $w_{N_1}, \dots, w_{N_K}$ sampled from a noise unigram distribution $P_n(w)$. The objective maximizes:

$$\mathcal{L}_{\text{NEG}} = \log \sigma \left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I} \right) + \sum_{k=1}^K \mathbb{E}_{w_{N_k} \sim P_n(w)} \left[\log \sigma \left(-\mathbf{u}_{w_{N_k}}^\top \mathbf{v}_{w_I} \right) \right], \quad (1.17)$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the logistic sigmoid function, $\mathbf{u}_{w_O}$ is the target output vector, and $\mathbf{u}_{w_{N_k}}$ is the output vector of the k -th noise word.

1.6.4 Full Step-by-Step Mathematical Derivation of SGNS Gradients

To demonstrate compliance with the *Mathematical Expansion Guidelines* (Section 1.6), we derive the exact analytical gradients of $\mathcal{L}_{\text{NEG}}$ with respect to all parameter vectors.

Recall the fundamental identity for the derivative of the log-sigmoid function:

$$\frac{d}{dz} \log \sigma(z) = \frac{1}{\sigma(z)} \sigma'(z) = \frac{1}{\sigma(z)} \sigma(z)(1 - \sigma(z)) = 1 - \sigma(z). \quad (1.18)$$

$$\begin{aligned} \frac{d}{dz} \log \sigma(-z) &= \frac{1}{\sigma(-z)} (-\sigma'(-z)) = \frac{1}{\sigma(-z)} (-\sigma(-z)(1 - \sigma(-z))) \\ &= -(1 - \sigma(-z)) = -\sigma(z), \end{aligned} \quad (1.19)$$

utilizing the symmetric identity $1 - \sigma(-z) = \sigma(z)$.

Gradient with Respect to Target Output Vector $\mathbf{u}_{w_O}$. The positive context vector $\mathbf{u}_{w_O}$ appears exclusively in the first term of Equation (1.17):

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{NEG}}}{\partial \mathbf{u}_{w_O}} &= \frac{\partial}{\partial \mathbf{u}_{w_O}} \left[\log \sigma \left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I} \right) \right] \\ &= \left[1 - \sigma \left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I} \right) \right] \frac{\partial \left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I} \right)}{\partial \mathbf{u}_{w_O}} \\ &= \left[1 - \sigma \left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I} \right) \right] \mathbf{v}_{w_I}. \end{aligned} \quad (1.20)$$

Gradient with Respect to Noise Vector $\mathbf{u}_{w_{N_k}}$. For any specific sampled negative token w_{N_k} , its vector appears only in its corresponding term in the summation:

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{NEG}}}{\partial \mathbf{u}_{w_{N_k}}} &= \frac{\partial}{\partial \mathbf{u}_{w_{N_k}}} \left[\log \sigma \left(-\mathbf{u}_{w_{N_k}}^\top \mathbf{v}_{w_I} \right) \right] \\ &= -\sigma \left(\mathbf{u}_{w_{N_k}}^\top \mathbf{v}_{w_I} \right) \frac{\partial (\mathbf{u}_{w_{N_k}}^\top \mathbf{v}_{w_I})}{\partial \mathbf{u}_{w_{N_k}}} \\ &= -\sigma \left(\mathbf{u}_{w_{N_k}}^\top \mathbf{v}_{w_I} \right) \mathbf{v}_{w_I}.\end{aligned}\quad (1.21)$$

Gradient with Respect to Center Input Vector $\mathbf{v}_{w_I}$. The center word vector $\mathbf{v}_{w_I}$ influences both the positive target term and all K negative noise terms simultaneously. Applying the linearity of differentiation:

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{NEG}}}{\partial \mathbf{v}_{w_I}} &= \frac{\partial}{\partial \mathbf{v}_{w_I}} \left[\log \sigma \left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I} \right) \right] + \sum_{k=1}^K \frac{\partial}{\partial \mathbf{v}_{w_I}} \left[\log \sigma \left(-\mathbf{u}_{w_{N_k}}^\top \mathbf{v}_{w_I} \right) \right] \\ &= \left[1 - \sigma \left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I} \right) \right] \mathbf{u}_{w_O} - \sum_{k=1}^K \sigma \left(\mathbf{u}_{w_{N_k}}^\top \mathbf{v}_{w_I} \right) \mathbf{u}_{w_{N_k}}.\end{aligned}\quad (1.22)$$

Intuitive Interpretation of the Gradients. Equations (1.20), (1.21), and (1.22) reveal the underlying mechanics of Negative Sampling:

- The term $[1 - \sigma(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I})]$ represents the *residual error* on the true word. If the model already predicts alignment $\sigma \approx 1$, the gradient vanishes. If alignment is low ($\sigma \approx 0$), the center vector $\mathbf{v}_{w_I}$ is pulled strongly in the direction of context vector $\mathbf{u}_{w_O}$.
- The term $-\sigma(\mathbf{u}_{w_{N_k}}^\top \mathbf{v}_{w_I})$ represents the penalty for falsely predicting noise words. If the model assigns high probability to a noise word ($\sigma \approx 1$), it receives a large negative gradient that aggressively repels $\mathbf{v}_{w_I}$ away from noise vector $\mathbf{u}_{w_{N_k}}$.
- Overall computational complexity drops to $\mathcal{O}(K \cdot d)$ operations per word pair, completely independent of vocabulary size V . Setting $K \in [5, 20]$ for small corpora and $K \in [2, 5]$ for massive corpora accelerates training by orders of magnitude.

Exam Insights: The 3/4 Exponent of the Noise Distribution

A staple exam question tests student knowledge of the noise distribution $P_n(w)$:

$$P_n(w) = \frac{U(w)^{3/4}}{\sum_{w'} U(w')^{3/4}}, \quad (1.23)$$

where $U(w)$ is the raw unigram empirical frequency in the corpus.

- *Why not use raw unigram frequency ($U(w)$)?* If we sample strictly proportional to raw counts, ultra-frequent stopwords (“the”, “is”, “of”) dominate the negative samples, while rare words are almost never sampled as negative distractors.
- *Why not use uniform distribution ($1/V$)?* Uniform sampling would select rare domain-specific words overwhelmingly often, ignoring linguistic frequency realities.
- *Why exponent $3/4 = 0.75$?* Raising frequencies to 0.75 serves as a non-linear regularizer: it compresses the probability gap between frequent and rare tokens.

For example, if word A occurs 1,000,000 times and word B occurs 1,000 times:

$$\frac{U(A)}{U(B)} = 1,000 \implies \frac{U(A)^{0.75}}{U(B)^{0.75}} = \frac{31,622}{177.8} \approx 177.8.$$

The probability ratio drops from 1,000 to 178, amplifying the probability that rare words appear as negative samples by over 5×.

1.7 Limitations of Word2Vec

Despite transforming NLP, classical Word2Vec architectures suffer from two major limitations (Slide 12):

1. Inability to Handle Out-of-Vocabulary (OOV) Tokens. Word2Vec assigns a fixed lookup row/column to each word observed during training. If a test document contains a word not present in the training vocabulary (e.g., technical jargon, a neologism like “*microservices*”, a typo like “*artificial*”, or an inflected morphological variant like “*unconstitutional*”), the model has no corresponding vector in $\mathbf{W}_{\text{in}}$. Standard workarounds (such as mapping all unknown words to a generic $\langle \text{UNK} \rangle$ vector) discard all semantic specificity.

2. Lack of Contextualization (Polysemy Collapse). Once training concludes, Word2Vec assigns exactly *one static vector* to each unique token string, completely regardless of downstream syntactic or semantic context:

Sentence A: “A **bat** is a nocturnal mammal that flies.”
Sentence B: “The player swung the baseball **bat** and hit a home run.”

In Sentence A, “bat” refers to a mammal; in Sentence B, “bat” refers to a piece of sports equipment. Because Word2Vec assigns a single parameter vector $\mathbf{v}_{\text{bat}}$, gradient descent averages both contexts into a single composite centroid vector situated halfway between zoology and athletics. It cannot dynamically contextualize meaning—a limitation that motivated contextualized representations (ELMo) and self-attention in Transformers (BERT, GPT).

1.8 FastText: Subword Character N-Gram Embeddings

To resolve the Out-of-Vocabulary pathology, Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov at Facebook AI Research developed **FastText** [?].

Rather than treating words as atomic indivisible symbols, FastText represents each word as a collection of character n -grams augmented with special boundary characters $\langle$ and $\rangle$:

- Boundary symbols $\langle$ and $\rangle$ demarcate prefixes and suffixes, distinguishing subwords like $\langle \text{her} \rangle$ (prefix) from $\text{her} \rangle$ (suffix) and $\text{her} \langle \text{ere} \rangle$ (internal stem).
- The full word itself is included within boundary tags $\langle w \rangle$ to retain whole-word semantics.

Exact Worked Example from Slide 13: The Word $\langle \text{where} \rangle$. For character trigrams ($n = 3$), the word “where” is segmented into:

$$\mathcal{G}_{\text{where}} = \{ \langle \text{wh}, \text{ whe}, \text{ her}, \text{ ere}, \text{ re} \rangle \} \cup \{ \langle \text{where} \rangle \}. \quad (1.24)$$

Each subword character n -gram $g \in \mathcal{G}_w$ is assigned its own learned embedding vector $\mathbf{z}_g \in \mathbb{R}^d$. The final vector representation of the word is computed as the sum of its constituent subword vectors:

$$\mathbf{v}_{\text{where}} = \sum_{g \in \mathcal{G}_{\text{where}}} \mathbf{z}_g = \mathbf{z}_{\langle \text{wh} \rangle} + \mathbf{z}_{\langle \text{whe} \rangle} + \mathbf{z}_{\langle \text{her} \rangle} + \mathbf{z}_{\langle \text{ere} \rangle} + \mathbf{z}_{\langle \text{re} \rangle} + \mathbf{z}_{\langle \text{where} \rangle}. \quad (1.25)$$

How FastText Solves OOV and Morphological Richness. If a completely novel word appears at test time (e.g., “*precomputation*”), FastText decomposes it into known character subwords ($\langle \text{pre}, \text{com}, \text{put}, \text{ati}, \text{ion} \rangle$). Summing the existing subword vectors synthesizes an accurate semantic representation for the unseen word without retraining. Furthermore, shared grammatical affixes (e.g., *-ation*, *-able*, *un-*) naturally share statistical strength across vocabulary entries.

1.9 Vector Geometry, Semantic Clustering, and Relational Offsets

When distributed representations are optimized over massive corpora, the resulting continuous vector space exhibits clean geometric structure (Slides 14 and 15):

1.9.1 Semantic Clustering via Principal Component Analysis (PCA)

FastText vectors are trained in high-dimensional Euclidean space ($d = 300$). Because visualizing 300 dimensions directly is impossible, we project the vectors onto a 2-dimensional plane using *Principal Component Analysis (PCA)*, which computes the orthogonal axes of maximal variance.

As demonstrated on Slide 14, when 300-dimensional FastText vectors for words belonging to three separate semantic categories are projected to 2D:

1. **Household Items:** *pen, pencil, table, chair, lamp.*
2. **Mammals:** *dog, cat, elephant, bat, lion.*
3. **Birds:** *eagle, sparrow, pigeon, hawk, owl.*

The words belonging to each category form tightly bound, mutually separated clusters in embedding space. Semantic relatedness maps directly to geometric proximity:

$$\cos(\mathbf{v}_{\text{dog}}, \mathbf{v}_{\text{cat}}) \gg \cos(\mathbf{v}_{\text{dog}}, \mathbf{v}_{\text{pencil}}). \quad (1.26)$$

1.9.2 Relational Linear Translations (Vector Analogies)

Remarkably, distributed vector spaces do not merely cluster synonyms; they capture linear semantic relationships. Semantic relations correspond to nearly constant displacement vectors (translations) across the vector space.

As shown on Slide 15:

$$\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}} \approx \vec{d}_{\text{capital_of}}. \quad (1.27)$$

The difference vector $\vec{d}_{\text{capital_of}}$ isolates the abstract semantic concept of “capital city of a country”. Applying this learned translation vector to a different country predicts its capital city with high geometric precision:

$$\mathbf{v}_{\text{Spain}} + \vec{d}_{\text{capital_of}} = \mathbf{v}_{\text{Spain}} + (\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}}) \approx \mathbf{v}_{\text{Madrid}}. \quad (1.28)$$

The most famous canonical instance of relational vector arithmetic is the gender-royalty parallelogram [?]:

$$\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} + \mathbf{v}_{\text{Woman}} \approx \mathbf{v}_{\text{Queen}}. \quad (1.29)$$

Figure 1.3 illustrates this geometric parallelogram in two dimensions.

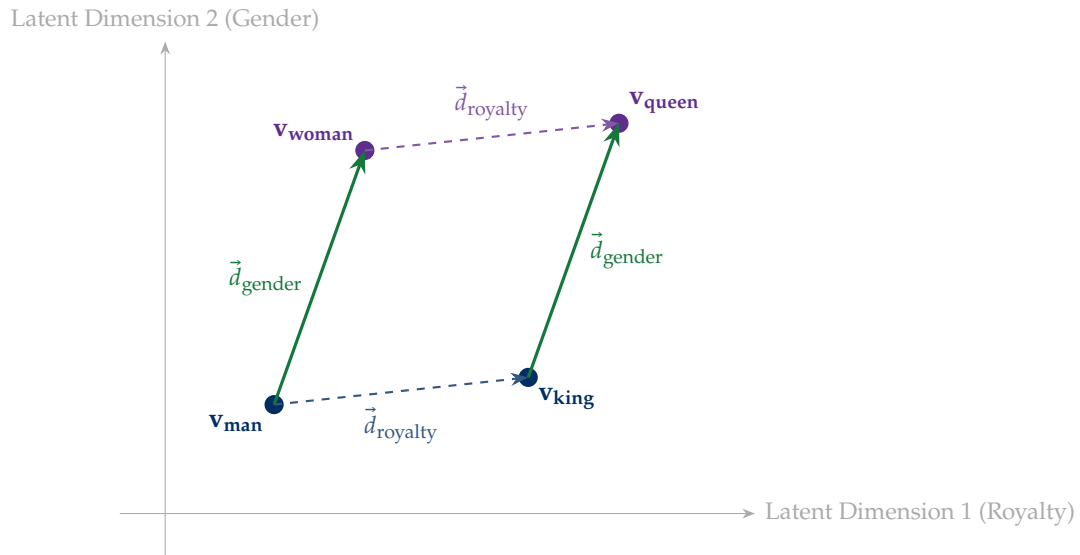


Figure 1.3: Geometric parallelogram in continuous embedding space demonstrating relational vector translations: $\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} \approx \mathbf{v}_{\text{Queen}} - \mathbf{v}_{\text{Woman}}$. The directional displacement $\vec{d}_{\text{gender}}$ isolates the feminine gender transformation, while $\vec{d}_{\text{royalty}}$ isolates the monarchical status offset.

External Deep Dive: Mikolov's 2013 Breakthrough and Word Analogy Benchmarks

Prior to Mikolov et al. [? ?], word embeddings were viewed as expensive auxiliary features for specialized neural taggers. Mikolov released pre-trained vectors trained on the Google News dataset (100 billion words, vocabulary size $V = 3 \times 10^6$, $d = 300$) alongside a formal **Word Analogy Benchmark** comprising 19,544 linguistic questions partitioned into syntactic analogies (e.g., *quick* : *quickly* :: *slow* : *slowly*) and semantic analogies (e.g., *Athens* : *Greece* :: *Rome* : *Italy*).

To answer the analogy query “*A* is to *B* as *C* is to ?”, the model searches for the vocabulary word w^* maximizing cosine similarity:

$$w^* = \arg \max_{w \in \mathcal{W} \setminus \{A, B, C\}} \cos(\mathbf{v}_w, \mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C). \quad (1.30)$$

SGNS achieved over 70% accuracy across the entire benchmark, establishing that simple log-linear architectures trained over massive corpora naturally discover linear manifold representations of human language.